

A 98%-accurate model that is completely useless

On paper, in pairs, with a pen. Compute precision and recall from printed confusion matrices, meet a model that is right 98% of the time and helps nobody, and choose a threshold from what mistakes cost.

Worksheet

Pairs

No laptop needed

Check cells afterwards

Before you start

1 · read the matrix

2 · four numbers by hand

3 · the 98% model

4 · choose the threshold

Before you start

Work on the printed sheet with your partner. The notebook version only holds check cells, to confirm your arithmetic after you have written it — not before.



By the end of this session you can

- Read every cell of a confusion matrix in the words of the problem.
- Compute accuracy, precision, recall and F1 by hand.
- Explain how class imbalance makes accuracy misleading.
- Choose a decision threshold from the costs of the two kinds of error.

QUESTION 1 · 6 MINUTES

read the matrix

1

A district health post predicts which of 200 booked patients will **miss** their appointment, so it can phone them the day before. On the test set:

	predicted: attends	predicted: misses
actually attended	169	1
actually missed	26	4

In the words of the clinic, not the words *true positive*: what does each of the four numbers count? Which of the two error cells costs a patient more?

QUESTION 2 · 8 MINUTES

four numbers by hand

2

Treat **misses** as the positive class. Compute, as fractions and then as decimals to three places:

- accuracy = (correct predictions) / 200
- precision = of the patients the model flagged, how many really missed
- recall = of the patients who really missed, how many the model flagged
- $F1 = 2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$

Then the baseline: the accuracy of a model that predicts **attends** for everyone.

check cell

PYTHON

```
# check cell – run only after both partners have written their answers
TN, FP, FN, TP = 169, 1, 26, 4
accuracy = (TP + TN) / (TN + FP + FN + TP)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1 = 2 * precision * recall / (precision + recall)
baseline = (TN + FP) / (TN + FP + FN + TP)
print(f"accuracy {accuracy:.3f} precision {precision:.3f} recall {recall:.3f} F1 {f1:.3f} baseline {baseline:.3f}")
```

what it should print

OUTPUT

```
accuracy 0.865 precision 0.800 recall 0.133 F1 0.229 baseline 0.850
```

QUESTION 3 · 5 MINUTES

the 98% model

3

A screening programme tests 1,000 people for a condition that 20 of them have. A model answers **healthy** for every single person.

1. What is its accuracy?
2. What is its recall for the condition?
3. Write one sentence a journalist could quote that is **true** and makes the model sound excellent — then one sentence that tells the truth about it.

check cell

PYTHON

```
# check cell
n, sick = 1000, 20
accuracy = (n - sick) / n
recall = 0 / sick
print("accuracy", accuracy, " recall", recall)
```

what it should print

OUTPUT

```
accuracy 0.98 recall 0.0
```

choose the threshold

4

The clinic model outputs a probability of missing. Here is what happens to precision and recall for the **misses** class as the cut-off moves:

given – the printed table

PYTHON

```
# given – how the table below was produced (synthetic appointment data, no real patients)
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score

rng = np.random.default_rng(12); n = 800
df = pd.DataFrame({"age": rng.integers(0, 91, n).astype(float), "distance_km":
np.round(rng.gamma(2.0, 2.0, n), 2),
                  "waiting_days": rng.integers(0, 31, n).astype(float), "sms_reminder":
rng.integers(0, 2, n),
                  "previous_no_shows": rng.poisson(0.5, n)})
logit = -3.0 + 0.08 * df.distance_km + 0.05 * df.waiting_days + 0.55 * df.previous_no_shows -
0.7 * df.sms_reminder
df["no_show"] = (rng.random(n) < 1 / (1 + np.exp(-logit))).astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(df.drop(columns="no_show"), df.no_show,
test_size=0.25, random_state=7, stratify=df.no_show)
pipe = Pipeline([("impute", SimpleImputer(strategy="median")), ("scale", StandardScaler()),
("model", LogisticRegression(max_iter=2000))].fit(X_tr, y_tr)
proba = pipe.predict_proba(X_te)[ :, 1]
rows = []
for t in [0.1, 0.2, 0.3, 0.4, 0.5]:
    pred = (proba ≥ t).astype(int)
    rows.append((t, int(pred.sum()), precision_score(y_te, pred, zero_division=0),
recall_score(y_te, pred)))
print(pd.DataFrame(rows, columns=["threshold", "patients flagged", "precision",
"recall"]).round(3).to_string(index=False))
```

what it should print

OUTPUT

threshold	patients flagged	precision	recall
0.1	117	0.222	0.867
0.2	46	0.391	0.600
0.3	18	0.444	0.267
0.4	9	0.556	0.167
0.5	5	0.800	0.133

A phone call costs the clinic about **200 FCFA** and five minutes of a nurse's time. A missed appointment wastes a consultation slot that another patient in the queue needed.

1. Which threshold would you choose, and why is 0.5 not automatically the answer?
2. What **one** extra number from the clinic would change your choice the most?